

Báo cáo Seminar T10 — Mutation Testing & Test Effectiveness

Nhóm 08 · CS423 / CSC15003 — Kiểm thử phần mềm (FIT@HCMUS)
GVHD (thực hành): ThS. Hồ Tuấn Thanh
Thành viên: 23127060 Ninh Văn Khải - 23127195 Trần Mạnh Hùng - 23127259 Nguyễn Tấn Thắng

Video demo (YouTube):
[Link Demo 1 \(Chính\): Hùng thực hiện trên Auth service](#)
[Link Demo 2 \(Phụ\): Khải thực hiện trên Order service](#)
[Link Demo 3 \(Phụ\): Thắng thực hiện trên Product service](#)

Mục lục

- 1. Chương 1: Tổng quan Seminar & Đặt vấn đề
- 2. Chương 2: Khảo sát & Chọn lựa công cụ (Stage S1)
- 3. Chương 3: Nền tảng lý thuyết Mutation Testing
- 4. Chương 4: Hướng dẫn cài đặt & Thiết lập EShop (User Guide - Stage S4)
- 5. Chương 5: Kết quả thực nghiệm & Hành trình nâng điểm Mutation Score
- 6. Chương 6: Phân tích chi tiết Surviving Mutants & Kỹ thuật diệt mutant
- 7. Chương 7: Quy trình AI-Augmented (Mutant-Guided AI Loop)
- 8. Chương 8: Failure Modes & Hạn chế của công cụ (Stage S4/T10)
- 9. Chương 9: Hoạt động lớp học "Kill the Mutant" (Stage S5/S6 - 25 phút)
- 10. Chương 10: Bộ hồ sơ AI Audit Pack Full (AI-02, AI-03, AI-04)
- 11. Chương 11: Bảng phân công & Đóng góp công việc (Project Contribution Statement)
- 12. Chương 12: Tài liệu tham khảo (References)

1. Chương 1: Tổng quan Seminar & Đặt vấn đề

Người phụ trách: 23127060 - Ninh Văn Khải

1.1 Đặt vấn đề — "Sự lừa dối" của Code Coverage (Nhóm có tham khảo nội dung từ Gemini Pro 3 - nội dung tham khảo 60% - tự chỉnh sửa 40%)

Trong thực tế phát triển phần mềm, **Code Coverage** (độ bao phủ mã nguồn) là chỉ số thường được sử dụng nhất để đo lường mức độ hoàn thiện của bộ kiểm thử. Tuy nhiên, Code Coverage chỉ đo lường xem *dòng code có được thực thi hay không*, chứ **không đảm bảo test suite có khả năng phát hiện ra lỗi logic, thuật ngữ chuyên ngành là sensitivity hay effectiveness của bộ test suit**.

Một bộ test suite có thể đạt **100% Code Coverage** nhưng vẫn hoàn toàn vô dụng nếu:

- Không có các câu lệnh khẳng định như là `assert` / `expect` để kiểm chứng kết quả.
- Câu lệnh `assert` quá sơ sài (chỉ kiểm tra HTTP status code status 200 mà bỏ qua nội dung response body).
- Bỏ sót các giá trị biên, ví dụ ta đổi `>=` thành `>` trong điều kiện kiểm tra nhưng bộ test case quên không cover branch này.

1.2 Triết lý Mutation Testing

Mutation Testing được ra đời để giải quyết triệt để hạn chế trên. Triết lý cốt lõi của chủ đề T10:

"Coverage lies. Mutation testing tells you if your tests can actually catch bugs." Đây là câu nói rất phổ biến trong giới phần mềm nêu bật lên tầm quan trọng của mutation trong việc kiểm thử test suite.

Ý tưởng là: Mutation Testing tự động cố ý chèn các lỗi nhỏ (gọi là các **mutants** - đột biến) vào mã nguồn thực tế (ví dụ: biến toán tử `+` thành `-`, đảo điều kiện `if`, xóa câu lệnh gán, đổi hằng số), sau đó chạy lại bộ test suite. Khi này mỗi test case cũ sẽ có 2 trường hợp xảy ra:

- Killed:** Tức là test case đó đã "bắt" được lỗi mà mutant gây ra (test case đó sẽ FAIL) → Bộ test nhạy bén, bắt được lỗi.

- **Survived:** Tất cả các test cases vẫn PASS → Bộ test yếu, có lỗi hổng vì các mutants vẫn còn sống sót. Khi này tester sẽ phải viết các bộ assert để bắt các survived mutants này.

1.3 Phạm vi áp dụng thực tế trên EShop

Nhóm 08 áp dụng kỹ thuật Mutation Testing trên ứng dụng web **EShop** (Node.js/Express + SQLite) tại thư mục `src/eshop-sut/backend`. Toàn bộ các bài kiểm thử Jest và cấu hình Stryker Mutator được thực thi trực tiếp trên mã nguồn thật của backend EShop (bao gồm các API Authentication, Cart, Coupon, Order và Product Admin). Tuy nhiên vì code của backend EShop có bug tồn tại sẵn nên nhóm chúng em tiếp tục dùng 1 phương pháp nữa là characterization testing.

Characterization testing hay còn gọi là kiểm thử đặc tính là phương pháp viết các bài test để ghi lại và "khóa" các hành vi thực tế của một hệ thống đã tồn tại. Thuật ngữ này được phổ biến bởi Michael Feathers trong cuốn sách Working Effectively with Legacy Code.^[1]

Vậy thì tại sao nhóm em cần dùng thêm phương pháp này ? Đó là bởi vì code gốc của Eshop vẫn có lỗi, (và trong thực tế kiểm thử phần mềm thì điều này thường xảy ra) -> Test suit báo đỏ và chúng em không thể chạy Mutation Testing. Vì vậy, phương pháp này giúp "tạm chấp nhận" tình trạng lỗi hiện tại của code và thực hiện Mutation Testing. Dĩ nhiên lỗi hiện tại sẽ được đưa vào trong Bugs List.

2. Chương 2: Khảo sát & Chọn lựa công cụ (Stage S1)

Người phụ trách: 23127195 - Trần Mạnh Hùng

Trong giai đoạn S1 (Tool Survey & Proposal), nhóm đã tiến hành khảo sát và so sánh các công cụ kiểm thử đột biến truyền thống và các công cụ AI hỗ trợ trên các tiêu chí: License, Chi phí, Độ tương thích với SUT EShop, Khả năng hỗ trợ của AI, và cộng đồng.

2.1 Bảng so sánh các công cụ truyền thống

Tiêu chí	Stryker Mutator	PIT (PITest)	mutmut	mull
Ngôn ngữ hỗ trợ	JavaScript, TypeScript, C#, Scala	Java, Kotlin	Python	C, C++
Test Runner	Jest, Mocha, Jasmine, Vitest	JUnit, TestNG	pytest	Catch2, gTest
Độ tương thích EShop	100% (Phù hợp native Node.js)	0% (Dành cho Java)	0% (Dành cho Python)	0% (Dành cho C++)
Kiểu tra về của báo cáo	HTML Interactive, JSON, Clear-text	HTML, XML	Terminal text	Mutation matrix
Giấy phép	Apache 2.0 (Miễn phí)	Apache 2.0 (Miễn phí)	MIT (Miễn phí)	BSD-3-Clause

2.2 Bảng so sánh các hướng công cụ AI-Augmented

Tiêu chí	ChatGPT / Claude (Prompt-based)	DiffBlue Cover	Pynguin + LLM
Phương thức	Prompt-based workflow	Auto-generated Java Unit Tests	Search-based + LLM repair
Ngôn ngữ hỗ trợ	Đa ngôn ngữ (JS/TS, Python, Java...)	Chỉ hỗ trợ Java	Chỉ hỗ trợ Python
Độ tương thích EShop	100% (Đọc hiểu JS diff & Jest)	0% (Chỉ chạy trên Java)	0%
Cài đặt / Dependency	Không cần cài plugin, dùng Web/API	Cần plugin Heavy IDE / CI	Cần môi trường Python

2.3 Lý do chọn lựa của Nhóm 08

- **Trước tiên chúng em sẽ bàn về các công cụ truyền thống**, đại diện là **Stryker Mutator** — Đây là tiêu chuẩn vàng về Mutation Testing cho hệ sinh thái JavaScript/TypeScript. Stryker hỗ trợ Jest runner, cung cấp Báo cáo HTML trực quan sắc nét và hỗ trợ chế độ phân tích theo từng test như là: `coverageAnalysis: "perTest"`. Tuy nhiên với người bắt đầu tiếp cận đến tools, chẳng hạn như nhóm 8 chúng em thì lần đầu sử dụng sẽ mất rất nhiều thời gian để biết xài cho đến khi thành thạo. Tuy nhiên với sự hỗ trợ của AI hiện nay thì điều đó đơn giản hơn nhiều, chúng ta có thể học nhanh hơn, làm quen với tools nhanh hơn. Cụ thể là ta có thể dùng **công cụ AI-Augmented**: ví dụ như các model LLM **ChatGPT / Claude / Gemini**. Do EShop viết bằng Node.js, các công cụ sinh test tự động như DiffBlue Cover không áp dụng được. Việc sử dụng ChatGPT/Claude theo quy trình prompt cho phép phân tích sâu diff ngữ nghĩa của mutant sống và gợi ý câu lệnh `expect(...)` chính xác.

3. Chương 3: Nền tảng lý thuyết Mutation Testing

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

3.1 Định nghĩa và Vòng đời của Mutant

Một **Mutant** là một phiên bản sửa đổi nhỏ của chương trình được tạo ra bằng cách áp dụng một **Mutation Operator** (toán tử đột biến). Các trạng thái của Mutant trong báo cáo Stryker gồm:

- 1. **Killed (Đã diệt):** Ít nhất một bài test bị thất bại khi chạy trên mutant → Kết quả mong muốn.
- 2. **Survived (Sống sót):** Mọi bài test đều vượt qua khi chạy trên mutant → Cần bổ sung test case/assertion.
- 3. **NoCoverage (Chưa bao phủ):** Không có bài test nào thực thi qua dòng code chứa mutant → Thiếu test case căn bản.
- 4. **Timeout (Quá thời gian):** Mutant gây ra vòng lặp vô tận khiến test bị hoãn → Được tính vào nhóm phát hiện được.
- 5. **CompileError / Runtime Exception (Lỗi):** Mutant gây lỗi cú pháp hoặc crash môi trường → Bị loại khỏi mẫu số tính điểm.

3.2 Khái niệm Equivalent Mutant (Đột biến tương đương)

Equivalent Mutant là những đột biến làm thay đổi cú pháp mã nguồn nhưng **không làm thay đổi mặt ngữ nghĩa hay logic** của chương trình.

- Ví dụ: Đổi vòng lặp `for (let i=0; i<10; i++)` thành `for (let i=0; i!=10; i++)`. Hai đoạn code này hoạt động hoàn toàn giống nhau trong mọi trường hợp.
- Hệ quả: Không có bài test nào có thể kill được Equivalent Mutant. Do đó, mục tiêu **Mutation Score 100% là không thực tế** trong công nghiệp; mức điểm lý tưởng là **70% – 80%**. Tuy nhiên trong phần chạy Mutation Testing của nhóm ở flow Product, mutation testing lại đạt 100%. Nhóm em không dám khẳng định bộ test bất tuyệt đối các đột biến, mà chỉ xin hiểu đúng là *"bắt được 100% trên các mutants, sau khi loại các mutant lỗi"*.

3.3 Công thức tính điểm Mutation Score

$$\text{Mutation Score} = \frac{\text{Killed} + \text{Timeout}}{\text{Total Valid Mutants}} \times 100\%$$

Trong đó:

- **Total Valid Mutants = Killed + Timeout + Survived + NoCoverage**
- **Killed:** Số mutant bị ít nhất 1 test case phát hiện (test FAIL).
- **Timeout:** Số mutant gây vòng lặp vô tận (được tính là đã bắt được).
- **Survived:** Số mutant sống sót (tất cả test vẫn PASS → cần bổ sung test).
- **NoCoverage:** Số mutant nằm ở dòng code chưa có test nào chạy qua.

4. Chương 4: Hướng dẫn cài đặt & Thiết lập EShop (User Guide - Stage S4)

Người phụ trách: 23127195 - Trần Mạnh Hùng

4.1 Môi trường và Lệnh cài đặt

Yêu cầu môi trường: **Node.js v22+**, **NPM v10+**.

```
# Chuyển vào thư mục backend của EShop
cd src\eshop-sut\backend

# Cài đặt các thư viện cần thiết cho Test và Stryker
npm install --save-dev jest @stryker-mutator/core @stryker-mutator/jest-runner supertest
```

Cập nhật package.json để khai báo các lệnh chạy kiểm thử:

```
"scripts": {
  "test": "jest --runInBand",
  "mutation:auth": "stryker run stryker.auth.config.mjs",
  "mutation:order": "stryker run stryker.order.config.mjs",
  "mutation:product": "stryker run stryker.product.config.mjs"
}
```

4.2 Cấu hình Stryker cô lập từng module (Scoped Configs)

Để tránh tình trạng Stryker nạp nhầm bài test của module khác gây xung đột database, nhóm đã thiết lập cấu hình Stryker riêng biệt cho từng module:

File stryker.order.config.mjs :

```
export default {
  packageManager: "npm",
  testRunner: "jest",
  reporters: ["html", "clear-text", "progress"],
  htmlReporter: { fileName: "reports/mutation-order/mutation.html" },
  mutate: ["services/orderService.js"],
  testFiles: ["tests/orders.api.test.js"],
  jest: { projectType: "custom", configFile: "jest.order.config.cjs", enableFindRelatedTests: false },
  coverageAnalysis: "perTest",
  thresholds: { high: 80, low: 60, break: 0 },
  timeoutMS: 10000,
};
```

File stryker.auth.config.mjs :

```
export default {
  packageManager: "npm",
  testRunner: "jest",
  reporters: ["html", "clear-text", "progress"],
  htmlReporter: { fileName: "reports/mutation-auth/mutation.html" },
  mutate: ["services/authService.js"],
  testFiles: ["__tests__/auth.api.test.js"],
  jest: { projectType: "custom", configFile: "jest.auth.config.cjs", enableFindRelatedTests: false },
  ignoreStatic: true,
  coverageAnalysis: "perTest",
  thresholds: { high: 80, low: 60, break: 0 },
  timeoutMS: 10000,
};
```

4.3 Giải pháp kỹ thuật xử lý crash SQLite Database (database.js)

Sự cố: Khi Stryker chạy nhiều worker song song (khoảng 15 worker), các worker cùng truy cập vào file vật lý database.sqlite trên Windows gây ra lỗi tranh chấp file SQLITE_BUSY và segfault 0xc0000005 .

Giải pháp: Chỉnh sửa database.js để tự động phát hiện môi trường kiểm thử Jest/Stryker và chuyển sang dùng cơ sở dữ liệu In-Memory (:memory:):

```
const sqlite3 = require("sqlite3").verbose();
const path = require("path");

// Khi chạy dưới dạng Jest worker hoặc sandbox Stryker (.stryker-tmp),
// dùng database :memory: riêng biệt cho từng luồng.
// Khi chạy production (node server.js), giữ nguyên 100% dùng database.sqlite thật.
const isStryker = __dirname.includes(".stryker-tmp") || process.env.JEST_WORKER_ID;
const dbPath = isStryker ? ":memory:" : path.resolve(__dirname, "database.sqlite");

const db = new sqlite3.Database(dbPath, (err) => {
  if (err) console.error("Could not connect to database", err);
});
```

4.4 Troubleshooting — Hướng dẫn khắc phục lỗi thường gặp

#	Lỗi thường gặp	Nguyên nhân	Cách khắc phục
1	SQLITE_BUSY: database is locked hoặc crash 0xc0000005 khi chạy Stryker	Nhiều worker Stryker cùng truy cập file database.sqlite vật lý trên Windows	Cập nhật database.js theo §4.3 để dùng :memory: khi phát hiện .stryker-tmp hoặc JEST_WORKER_ID
2	No tests were found khi chạy npm run mutation:auth	testMatch hoặc rootDir trong file Jest config bị resolve sai trong sandbox .stryker-tmp/ của Stryker	Dùng rootDir: __dirname thay vì đường dẫn tương đối; cấu hình testMatch: ["**/auth.api.test.js"] thay vì <rootDir>/...
3	Stryker chạy rất lâu (hàng giờ) hoặc báo table already exists	Thiếu thuộc tính configFile và testFiles trong stryker.*.config.mjs → Stryker nạp tất cả test của mọi module	Tạo cấu hình Stryker riêng biệt cho từng module (xem §4.2) với jest.configFile và testFiles cô lập
4	npm test pass nhưng npm run mutation:order vẫn crash	process.env.JEST_WORKER_ID không tồn tại trong giai đoạn Dry-Run của Stryker (chạy Jest in-band đơn luồng)	Bổ sung kiểm tra __dirname.includes('.stryker-tmp') ngoài JEST_WORKER_ID (xem §4.3)
5	Score Mutation hiển thị rất cao (~93%) nhưng số Errors lớn (>100)	Mutant bị lỗi crash bị loại khỏi mẫu số → Score bị thổi phồng ảo	Kiểm tra cột Errors trong báo cáo; nếu >10% tổng mutant, cần sửa config trước khi tin số liệu
6	npm install báo npm audit vulnerabilities	Dependency tree của EShop backend có các thư viện cũ	Không chạy npm audit fix trong quá trình thí nghiệm để tránh thay đổi baseline; ghi nhận cảnh báo vào báo cáo

5. Chương 5: Kết quả thực nghiệm & Hành trình nâng điểm Mutation Score

Người phụ trách: 23127060 - Ninh Văn Khải & 23127259 - Nguyễn Tấn Thắng

Bộ test suite hiện tại của nhóm trên backend EShop đã đạt **52/52 tests PASSED** (4 test suites) trên toàn hệ thống.

5.1 Bảng tổng hợp hành trình nâng điểm trên toàn Backend EShop

Module EShop	Số Test Jest	Baseline Score	Improved Score	Final Score	Killed / Valid Mutants	Trạng thái
Cart / Coupon / Order (orderService.js)	21	54.29%	92.57%	99.43%	163 / 175 (11 Timeout)	Đạt chuẩn
Authentication / User (authService.js)	19	31.30%	59.83%	87.16%	80 / 91 (11 Survived)	Đạt chuẩn
Product / Admin APIs (productService.js)	18	9.66%	100.00%	100.00%	165 / 165 (11 Timeout)	Đạt tối đa tuyệt đối

5.2 Diễn giải chi tiết hành trình nâng điểm (Order Module) (AI hỗ trợ viết báo cáo 40%, Khả thực hiện review, chạy kiểm chứng 60% và chỉnh sửa nếu có sai sót)

Người phụ trách: 23127060 - Ninh Văn Khải

1. **Giai đoạn Baseline (5 test — Score 54.29%):** Bộ test ban đầu chỉ kiểm tra status code `res.status` là `200 OK` . Do đó, khi Stryker thay đổi nội dung bên trong logic trả về (xóa giỏ hàng, đảo ngược filter user), mọi test đều vẫn PASS → 6 surviving mutants và 69 no-coverage.
2. **Giai đoạn Improved (21 test — Score 84.21%):** Bổ sung các câu lệnh assert chi tiết vào response body (như `expect(res.body).toEqual([])`), kiểm tra phân quyền sở hữu đơn hàng và các trường hợp lỗi biên coupon. Phát hiện và xử lý **lỗi ô nhiễm trạng thái (State Pollution)** giữa các test coupon bằng cách cô lập tài khoản người dùng riêng cho từng test.
3. **Giai đoạn Final (21 test — Score 92.57%):** Sửa dứt điểm lỗi cấu hình Stryker scope khiến các mutant bị lỗi (error) không còn bị tính sai vào mẫu số. Đạt 154 killed + 8 timeout trên tổng số 175 mutants hợp lệ.

Phát hiện bug thật trong EShop: Trong quá trình phân tích coupon, em (Ninh Văn Khải) phát hiện logic tính toán giảm giá phần trăm trong mã nguồn gốc bị sai: code gốc dùng `1 - discount_value` thay vì `discount_value / 100` để tính tỷ lệ phần trăm. Nhóm quyết định **giữ nguyên logic thật** (dùng characterization testing) và viết assertion kiểm chứng đúng hành vi hiện tại (không tự ý sửa code gốc) — đây là minh chứng rõ ràng rằng mutation testing có thể giúp phát hiện bug thật trong sản phẩm.

Danh sách 21 test cases của Order module:

#	Test case	Mục đích
1	GET /api/cart returns empty array initially	Đảm bảo logic tạo mảng nếu chưa có
2	POST /api/cart adds item to cart	Kiểm tra giá trị lưu vào cart
3	POST /api/checkout creates an order	Test flow tạo order
4	GET /api/orders/my-orders returns only user's orders	Phân quyền truy xuất order của đúng user
5	GET /api/orders/:id returns order details	Lấy order by id (success)
6	GET /api/orders/:id returns 404 for unknown order	Lấy order by id (not found)
7	PUT /api/orders/:id/cancel cancels the order successfully	Cancel order (success)
8	PUT /api/orders/:id/cancel fails if order already canceled	Ngăn hủy order đã bị hủy/giao
9	PUT /api/orders/:id/cancel returns 404 for other user's order	Bảo mật quyền hủy order theo user
10	PUT /api/orders/:id/cancel handles DB error gracefully	Xử lý lỗi DB
11	getMyOrders handles DB error gracefully	Xử lý lỗi DB
12	POST /api/apply-coupon requires code	Validation coupon trống
13	POST /api/apply-coupon handles unknown or inactive code	Validation code không tồn tại
14	POST /api/apply-coupon handles expired coupon	Logic ngày hết hạn
15	POST /api/apply-coupon rejects if total < min_order_amount	Logic giá trị tối thiểu đơn hàng
16	POST /api/apply-coupon applies valid percent coupon	Tính toán giảm giá theo phần trăm
17	POST /api/apply-coupon applies valid fixed coupon	Tính toán giảm giá theo mức cứng
18	POST /api/coupon-usage saves usage	Đếm số lần sử dụng
19	POST /api/apply-coupon tracks max usage per user limit	Ràng buộc sử dụng tối đa
20	POST /api/apply-coupon allows other users to use coupon	Kiểm tra tính chia sẻ coupon
21	POST /api/apply-coupon handles fixed coupon with user_id	Logic chi tiết discount

5.3 Diễn giải chi tiết hành trình nâng điểm (Auth Module)

Người phụ trách: 23127195 - Trần Mạnh Hùng

- Giai đoạn Baseline (4 test — Score 31.30%):** Test ban đầu chỉ bao gồm 4 case cơ bản: đăng ký user, đăng nhập đúng, đăng nhập sai mật khẩu, profile không có token. Kết quả: Killed 32, Timeout 4, Survived 11, NoCoverage 68.
- Giai đoạn Improved (9 test — Score 59.83%):** Bổ sung: đăng ký trùng email, login email không tồn tại, khóa tài khoản sau nhiều lần sai password, lỗi database khi login, decode JWT payload, profile với token hợp lệ. Survived giảm từ 11 xuống 0, nhưng NoCoverage vẫn còn 47 vì 3 endpoint (forgotPassword , resetPassword , updateCurrentUser) chưa có test.
- Giai đoạn Final (19 test — Score 87.16%):** Bổ sung Part B gồm 10 test cases mới bao phủ 3 endpoint còn thiếu: forgot-password (email tồn tại/không tồn tại), reset-password (token đúng/sai/email sai), update profile (có/không role, lỗi DB). Kết quả sạch: Killed 80, Survived 11, NoCoverage 0.

Danh sách test cases của Auth module:

#	Test case	Mục đích
1	registers a user and returns a useful response body	Register thành công

#	Test case	Mục đích
2	rejects duplicate registration with the same email	Cover nhánh lỗi DB unique constraint
3	logs in with a registered user and returns a JWT containing user data	Login thành công + kiểm tra JWT payload
4	rejects login with a wrong password	Login sai password
5	rejects login for an unknown email	Cover nhánh !user
6	locks a user after repeated wrong passwords	Logic login_attempts + locked_until
7	returns database error when login query fails	Cover nhánh err của db.get
8	requires a token for current user profile	Middleware khi thiếu token
9	reads current user profile with a valid token	Middleware token hợp lệ
10	returns 200 and a resetToken for an existing user	Forgot-password thành công
11	returns 404 when the email is not registered	Forgot-password email không tồn tại
12	resets the password with a valid token and allows re-login	Reset-password full flow
13	returns 400 when the token is wrong	Reset-password token sai
14	returns 400 when the email does not match	Reset-password email sai
15	returns 401 when no token is provided	Update profile thiếu token
16	updates profile without role field and returns 200	Update profile (nhánh không có role)
17	updates profile WITH role field	Update profile (nhánh có role)
18	returns 500 when the DB update fails (unit-level)	Cover nhánh lỗi DB update
19	verifies updated profile is persisted	Xác nhận dữ liệu lưu đúng

5.4 Diễn giải chi tiết kết quả (Product/Admin Module)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Module Product/Admin đạt **Mutation Score 100%** (Killed 165 + Timeout 11, Survived 0, NoCoverage 0) trên 185 mutants tổng cộng. Kết quả này đạt được nhờ Thắng áp dụng ngay các bài học từ module Auth và Order: viết assertion đầy đủ cho status code, response body, ID và message ngay từ đầu thay vì chỉ check status 200.

Phạm vi kiểm thử — 8 handler được tách từ `server.js` :

Handler	API	Loại test
listProducts	GET /api/products	Danh sách + tìm kiếm
getProductById	GET /api/products/:id	Chi tiết + not found
createProduct	POST /api/products	Tạo mới + lỗi DB
updateProduct	PUT /api/products/:id	Cập nhật + kiểm tra không ảnh hưởng record khác
deleteProduct	DELETE /api/products/:id	Xóa + xác nhận đã xóa
importProducts	POST /api/admin/import-products	Import CSV/JSON, lỗi từng dòng
listAdminOrders	GET /api/admin/orders	Thứ tự + tên người đặt
updateAdminOrderStatus	PUT /api/admin/orders/:id/status	State machine (pending→confirmed→shipping→delivered/canceled)

Kỹ thuật đạt 100%: Dùng fake database xác định (deterministic) trong file test thay vì SQLite thật → tránh hoàn toàn xung đột multi-worker; assert đầy đủ status code, response body, ID, message, thứ tự kết quả; bao phủ cả nhánh thành công lẫn nhánh lỗi DB.

Hành vi bất thường của SUT được test ghi nhận:

Trong quá trình viết test, em phát hiện và **ghi nhận trung thực** các hành vi bất thường của EShop mà mutation score 100% không phản ánh:

#	Hành vi bất thường	Rủi ro tiềm ẩn
1	Product ID chặn trả <code>price</code> dạng chuỗi (<code>"30000000"</code>), ID lẻ trả dạng số (<code>28000000</code>)	Inconsistency kiểu dữ liệu API
2	Product không tồn tại trả HTTP <code>200</code> với object rỗng thay vì <code>404</code>	Frontend không phân biệt được success/error
3	Các route tạo/sửa/xóa product hiện chưa yêu cầu token admin	Lỗi hỏng phân quyền nghiêm trọng
4	Import products chấp nhận insert một phần thay vì rollback toàn bộ	Dữ liệu bị partial insert khi có lỗi
5	Order ở trạng thái <code>canceled</code> vẫn có thể chuyển sang <code>delivered</code>	Sai logic state machine
6	Search query ghép trực tiếp từ khóa vào chuỗi SQL	Rủi ro SQL Injection

Kinh nghiệm nhóm rút ra: Mutation Score 100% **không** có nghĩa phần mềm không có lỗi. Mutation testing đo *khả năng phát hiện lỗi cú pháp* của bộ test, chứ không đo *tính đúng đắn của logic nghiệp vụ*.

6. Chương 6: Phân tích chi tiết Surviving Mutants & Kỹ thuật diệt mutant

Người phụ trách: 23127060 - Ninh Văn Khải

Dưới đây là 5 ví dụ tiêu biểu về các Mutant bị sống sót (Surviving Mutants) trong mã nguồn EShop thật và kỹ thuật viết assertion bổ sung để tiêu diệt chúng:

Bảng phân tích Surviving Mutants & Kỹ thuật diệt

#	Mutant Location	Code gốc	Code bị Mutate	Nguyên nhân Mutate Survived
1	orderService.js (Cart)	<code>userCarts[userId] = []</code>	<code>userCarts[userId] = ["Stryker was here"]</code>	Test ban đầu chỉ check status <code>200</code> , không kiểm tra giỏ hàng có rỗng thật không.
2	orderService.js (Order Filter)	<code>filter(o => o.user_id === req.user.id)</code>	<code>filter(o => o.user_id !== req.user.id)</code>	Test chưa kiểm tra thuộc tính <code>user_id</code> của danh sách đơn hàng trả về.
3	orderService.js (Coupon Limit)	<code>if (usage_count >= max_uses)</code>	<code>if (usage_count > max_uses)</code>	Test chưa bao phủ điểm ranh giới (Boundary Value) tại đúng ngưỡng <code>usage_count = max</code> .
4	authService.js (Login Retry)	<code>user.login_attempts + 1</code>	<code>user.login_attempts - 1</code>	Test chỉ kiểm tra 1 lần nhập sai password, chưa test chuỗi lần sai.
5	authService.js (JWT Payload)	<code>jwt.sign({ id, role }, SECRET)</code>	<code>jwt.sign({}, SECRET)</code>	Test chỉ kiểm tra token tồn tại (<code>expect(token).toBeDefined()</code>).

7. Chương 7: Quy trình AI-Augmented (Mutant-Guided AI Loop)

Người phụ trách: 23127060 - Ninh Văn Khải

Nhóm áp dụng quy trình **Mutant-Guided AI Loop** nhằm phối hợp giữa khả năng phân tích của AI (ChatGPT/Claude) và tính kỷ luật kiểm chứng của con người.

[Chạy Stryker] —> [Trích xuất Surviving Mutant Diff] —> [Prompt cho ChatGPT/Claude]
|
[Lưu Báo cáo] <— [Tự chạy Jest/Stryker Verify] <— [AI gợi ý Test/Assertion]

7.1 Nguyên tắc của nhóm em

AI chỉ đóng vai trò gợi ý — Con người luôn tự chạy lại Jest và Stryker trên máy để kiểm chứng thực tế.

7.2 Log Prompt tương tác thực tế

Log xử lý các Mutant Survived liên quan đến lỗi Database:

- **Khải:** "Tôi dùng mutation testing nhưng hiện tại có 1 trường hợp mutation còn survived như trên. Giúp tôi code để bắt mutation này ?"
- **AI:** "Trường hợp này liên quan đến nhánh xử lý lỗi của Database. Bạn cần dùng `\jest.spyOn\` để giả lập (mock) lỗi cho các hàm của SQLite (`'db.get'`, `'db.run'`, `'db.all'`). Dưới đây là ví dụ code để bắt mutation này:"

```
test("keeps existing cart items when adding multiple items", async () => {
  const { token } = await registerAndLogin("cart-multi-items");
  const item1 = { id: 1, name: "Item 1", price: 10000, quantity: 1 };
  const item2 = { id: 2, name: "Item 2", price: 20000, quantity: 1 };

  // 1. Thêm món thứ nhất
  const addResponse1 = await request(app)
    .post("/api/cart")
    .set("Authorization", `Bearer ${token}`)
    .send(item1);
  expect(addResponse1.status).toBe(200);

  // 2. Thêm món thứ hai (Mutant sẽ xóa mất item1 ở bước này!)
  const addResponse2 = await request(app)
    .post("/api/cart")
    .set("Authorization", `Bearer ${token}`)
    .send(item2);
  expect(addResponse2.status).toBe(200);

  // 3. Lấy giỏ hàng và assert đầy đủ cả 2 sản phẩm
  const cartResponse = await request(app)
    .get("/api/cart")
    .set("Authorization", `Bearer ${token}`);

  expect(cartResponse.status).toBe(200);
  expect(cartResponse.body).toEqual([item1, item2]); // Mutant sẽ chỉ có [item2] -> Test FAIL -> Mutant KILLED!
});
```

- **Khải:** "tại sao ko kill được mutant ? Có phải là equivalent mutant ?"
- **AI:**
"Đây không phải là equivalent mutant. Nguyên nhân không kill được mutant là do bộ test của bạn đang thiếu các test case bao phủ tình huống thạc

```

test("returns database error when getMyOrders fails", async () => {
  const { token } = await registerAndLogin("orders-db-error");
  const spy = jest.spyOn(db, "all").mockImplementationOnce((sql, params, callback) => {
    callback(new Error("forced getMyOrders failure"));
  });

  const response = await request(app).get("/api/orders/my-orders").set("Authorization", `Bearer ${token}`);
  spy.mockRestore();

  expect(response.status).toBe(500);
  expect(response.body).toEqual({ error: "forced getMyOrders failure" });
});

test("returns database errors when getOrderById fails", async () => {
  const spy = jest.spyOn(db, "get").mockImplementationOnce((sql, params, callback) => {
    callback(new Error("forced getOrderById failure"));
  });

  const response = await request(app).get("/api/orders/1");
  spy.mockRestore();

  expect(response.status).toBe(500);
  expect(response.body).toEqual({ error: "forced getOrderById failure" });
});

test("returns database errors when cancelOrder select or update fails", async () => {
  const user = await registerAndLogin("cancel-db-error");
  const order = await createOrder(user.token);

  // 1. Test cancelOrder SELECT failure
  const selectSpy = jest.spyOn(db, "get").mockImplementationOnce((sql, params, callback) => {
    callback(new Error("forced cancel select failure"));
  });

  const selectResponse = await cancelOrder(user.token, order.id);
  selectSpy.mockRestore();

  expect(selectResponse.status).toBe(500);
  expect(selectResponse.body).toEqual({ error: "forced cancel select failure" });

  // 2. Test cancelOrder UPDATE failure
  const runSpy = jest.spyOn(db, "run").mockImplementation((sql, params, callback) => {
    if (sql.includes("UPDATE orders SET status = ?")) {
      callback(new Error("forced cancel update failure"));
    }
  });

  const updateResponse = await cancelOrder(user.token, order.id);
  runSpy.mockRestore();

  expect(updateResponse.status).toBe(500);
  expect(updateResponse.body).toEqual({ error: "forced cancel update failure" });
});

test("returns database errors when applyCoupon queries fail", async () => {
  const user = await registerAndLogin("apply-coupon-db-error");

  // 1. Test applyCoupon SELECT coupon failure
  const couponSpy = jest.spyOn(db, "get").mockImplementationOnce((sql, params, callback) => {
    callback(new Error("forced coupon select failure"));
  });

  const couponResponse = await request(app)
    .post("/api/apply-coupon")
    .send({ code: "SAVE10", total_amount: 500000 });
  couponSpy.mockRestore();

  expect(couponResponse.status).toBe(500);

```

```
expect(couponResponse.body).toEqual({ error: "forced coupon select failure" });

// 2. Test applyCoupon SELECT usage count failure
const originalGet = db.get.bind(db);
const usageSpy = jest.spyOn(db, "get").mockImplementation((sql, params, callback) => {
  if (sql.includes("COUNT(*)")) {
    callback(new Error("forced usage count failure"));
  } else {
    originalGet(sql, params, callback);
  }
});

const usageResponse = await request(app)
  .post("/api/apply-coupon")
  .send({ code: "SAVE10", total_amount: 500000, user_id: user.id });

usageSpy.mockRestore();

expect(usageResponse.status).toBe(500);
expect(usageResponse.body).toEqual({ error: "forced usage count failure" });
});
```

- **Kiểm chứng:** Khai đã bổ sung các test case mô phỏng lỗi database cho các API của Order. Kết quả: tiêu diệt thành công toàn bộ các mutant lỗi DB cuối cùng, đưa Mutation Score của module Order đạt đỉnh **99.43%** (163/175 mutants bị killed).
- **Claim:** Phần quá trình phân tích và diệt các mutant khó (survived mutants) liên quan đến lỗi Database này đã được em mô tả và thao tác thực hành rất kỹ trong video YouTube nộp kèm. (DEMO WORKFLOW)](<https://www.youtube.com/watch?v=23PBaJsPs0g>)

8. Chương 8: Failure Modes & Hạn chế của công cụ (Stage S4/T10)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Theo yêu cầu bắt buộc của chủ đề T10, báo cáo phải chỉ ra ít nhất 3 cách công cụ Stryker hoặc AI đưa ra kết quả sai lệch hoặc gây hiểu lầm cho người dùng.

8.1 Failure Modes của công cụ truyền thống (Stryker Mutator)

1. Lỗi Xung đột File Database Vật lý trên Windows (SQLITE_BUSY):
- *Hiện tượng:* Khi Stryker chạy mặc định với nhiều worker, các tiến trình con cùng mở file database.sqlite gây crash 0xc0000005 và báo điểm ảo thấp do mutant bị đánh dấu là Error.
 - *Khắc phục:* Phải can thiệp vào database.js để ép Stryker dùng In-Memory DB (:memory:).
2. Lỗi Sai lệch Scope Cấu hình (enableFindRelatedTests):
- *Hiện tượng:* Nếu thiếu testFiles hoặc configFile riêng, Stryker tự động nạp toàn bộ bài test của các module khác → thời gian chạy kéo dài hàng giờ và báo lỗi thiếu bằng.
3. Giới hạn với Equivalent Mutants:
- *Hiện tượng:* Stryker đảo ngược các dòng log hoặc các phép toán tương đương ngữ nghĩa. Người dùng không thể diệt được các mutant này và buộc phải chấp nhận điểm Mutation Score < 100%.

8.2 Failure Modes của công cụ AI (ChatGPT / Claude)

#	Loại lỗi AI (Failure Mode)	Mô tả chi tiết sai sót của AI	Cách nhóm phát hiện & Khắc phục
1	Factual Error (Lỗi sai cú pháp/thực tế)	AI gợi ý câu lệnh expect(res.body).toBe([]) . Trong Jest, .toBe() so sánh bằng object.is (reference equality), dẫn đến so sánh mảng mới [] luôn luôn FAIL.	Kiểm tra cú pháp Jest → Sửa thành .toEqual([]) kiểm tra deep equality.
2	Missing Edge Cases (Bỏ sót trường hợp)	Khi gợi ý test cho boundary coupon, AI chỉ tạo test case đơn luồng. AI hoàn toàn bỏ sót các trường hợp Concurrency / Race Condition	Phát hiện khi audit → Ghi nhận giới hạn của AI trong việc phát hiện lỗi đồng thời.

#	Loại lỗi AI (Failure Mode)	Mô tả chi tiết sai sót của AI	Cách nhóm phát hiện & Khắc phục
	<i>biên</i>)	(nhiều request gửi cùng 1 millisecond).	
3	Silent Assumptions (Giả định ngầm sai lầm)	AI giả định ngầm rằng database tự động dọn dẹp (reset) sau mỗi test case, không tạo hàm <code>beforeEach()</code> làm sạch dữ liệu → gây ra lỗi State Pollution giữa các test.	Thêm cleanup hooks <code>beforeEach()</code> và cô lập email/user ngẫu nhiên.
4	Over-confident Statement (Tự tin thái quá)	AI khẳng định: " <i>Câu lệnh assert này sẽ 100% giúp bộ test của bạn bulletproof (hoàn hảo tuyệt đối).</i> "	Đánh giá phản biện: Diệt được mutant không đồng nghĩa với mã nguồn không có lỗi kiến trúc/bảo mật (như IDOR).

8.3 Nhánh code chưa được bảo vệ (Ghi nhận trung thực)

Trong module Order, hiện còn **6 survived + 7 no-coverage** tập trung ở các nhánh bắt lỗi Database Callback `if (err) return res.status(500).json({ error: err.message })` . Nhóm ghi nhận trung thực hạn chế này.

9. Chương 9: Hoạt động lớp học "Kill the Mutant" (Stage S5/S6 - 25 phút)

Người phụ trách: 23127060 - Ninh Văn Khải

Nhóm thiết kế hoạt động tương tác 25 phút dành cho khán giả dưới lớp học theo đúng format của tài liệu T10 Briefing. Vì đây là báo cáo nên nhóm chỉ đưa sơ qua phần mutants. Seminar sẽ có file .md chi tiết cho các bạn trong lớp.

9.1 Kế hoạch tổ chức (Timeline 25 phút)

- **0:00 – 0:03 (3 phút):** Người điều phối (Facilitator) trình bày 5 đoạn Mutant Diffs sống sót trích từ EShop.
- **0:03 – 0:13 (10 phút):** Các nhóm khán giả thảo luận và viết câu lệnh assertion (Jest/pseudocode) vào Worksheet.
- **0:13 – 0:18 (5 phút):** Đổi bài chéo giữa các nhóm khán giả để đánh giá phản biện.
- **0:18 – 0:22 (4 phút):** Facilitator chạy thử các assertion trên môi trường EShop sandbox thật và tính điểm diệt mutant.
- **0:22 – 0:25 (3 phút):** Đại diện nhóm thắng cuộc giải thích tư duy thiết kế assertion.

9.2 Đề bài 5 Mutant Diffs cho Khán giả

```
// MUTANT 1 (Cart Module)
- if (!userCarts[userId]) userCarts[userId] = [];
+ if (!userCarts[userId]) userCarts[userId] = ["Stryker was here"];

// MUTANT 2 (Order Module)
- const userOrders = orders.filter(o => o.user_id === req.user.id);
+ const userOrders = orders.filter(o => o.user_id !== req.user.id);

// MUTANT 3 (Coupon Boundary)
- if (coupon.usage_count >= coupon.max_uses) {
+ if (coupon.usage_count > coupon.max_uses) {

// MUTANT 4 (Auth Lockout)
- const newAttempts = user.login_attempts + 1;
+ const newAttempts = user.login_attempts - 1;

// MUTANT 5 (JWT Payload)
- const token = jwt.sign({ id: user.id, role: user.role }, SECRET);
+ const token = jwt.sign({}, SECRET);
```

9.3 Đáp án chuẩn

1. **Mutant 1:** `expect(res.body).toEqual([])` hoặc `expect(res.body.length).toBe(0)` .
2. **Mutant 2:** `res.body.forEach(order => expect(order.user_id).toBe(currentUser.id))` .

3. **Mutant 3:** Tạo coupon có `usage_count = max_uses` , gọi API `apply` → `expect(res.status).toBe(400)` .
4. **Mutant 4:** Gọi login sai password 3 lần → `expect(res.body.error).toContain("Tài khoản đã bị khóa")` .
5. **Mutant 5:** `const decoded = jwt.decode(res.body.token); expect(decoded.id).toBeDefined()` .

10. Chương 10: Bộ hồ sơ AI Audit Pack Full (AI-02, AI-03, AI-04)

Người phụ trách: 23127060 - Ninh Văn Khải

10.1. Nhật ký 6 sự cố AI làm sai & cách nhóm tự khắc phục

#	AI gợi ý / làm gì	Triệu chứng thất bại	Nguyên nhân gốc	Cách nhóm tự khắc phục	Trạng thái
1	Sinh <code>stryker.order.config.mjs</code> thiếu <code>scope</code>	Crash hàng loạt: <code>SQLITE_BUSY</code> , <code>table already exists</code> ; 111 mutant errors	Config thiếu <code>configFile</code> nên Stryker nạp bài test của mọi module khác	Thêm <code>testFiles</code> và <code>configFile: "jest.order.config.cjs"</code>	✅ Đã fix
2	Báo Mutation Score 93.15%	Điểm số bị thổi phồng ảo	111 mutant lỗi bị AI loại khỏi mẫu số tính điểm	Chạy lại bản scoped sạch, đo điểm thật 92.57%	✅ Đã fix
3	Gợi ý assertion để kill mutant	Assertion sai cú pháp (<code>.toBe([])</code>)	AI không chạy code thật nên không biết cơ chế <code>reference equality</code> của Jest	Kiểm tra cú pháp, sửa thành <code>.toEqual([])</code>	✅ Đã fix
4	Sinh Jest config dùng <code><rootDir>/tests/...</code>	Jest báo <code>No tests were found</code> rồi thoát sớm	<code><rootDir></code> bị resolve sai trong sandbox <code>.stryker-tmp/</code>	Đổi thành <code>rootDir: __dirname + roots: ["<rootDir>/__tests__"]</code>	✅ Đã fix
5	AI ước lượng score ~84%	Số ước lượng lệch với thực tế	AI đoán từ dữ liệu báo cáo cũ	Chạy thật ra 92.57% (162/175)	✅ Đã fix
6	Sinh <code>stryker.auth.config.mjs</code> gây crash Auth	Crash <code>0xc0000005</code> , <code>no such table: products</code>	15 worker cùng ghi đè lên file <code>database.sqlite</code> vật lý	Cập nhật <code>database.js</code> : kiểm tra <code>.stryker-tmp</code> để dùng <code>:memory:</code>	✅ Đã fix

10.2. [AI-02] Báo cáo kiểm định AI (AI Audit Report)

Chủ đề seminar: T10 - Mutation Testing & Test Effectiveness

Người kiểm định: 23127060 - Ninh Văn Khải (Nhóm 08)

1. Bối cảnh và mô tả đối tượng kiểm định

Trong khuôn khổ Seminar T10 về Mutation Testing, nhóm em sử dụng Stryker Mutator trên dự án backend EShop, tập trung vào module `orderService.js` . Ở lần chạy baseline, nhiều mutant sống sót vì bộ test Jest ban đầu chỉ kiểm tra mã trạng thái HTTP 200 mà không xác minh nội dung response trả về. Để cải thiện mutation score, em đã dùng một mô hình ngôn ngữ lớn (ChatGPT/Claude) để sinh các assertion Jest nhằm tiêu diệt những mutant còn sống (ví dụ: mutant khởi tạo mảng và mutant biên của logic sử dụng coupon).

Đối tượng được kiểm định trong báo cáo này là **các phần giải thích và các assertion Jest do AI đề xuất** trong quá trình nhóm em đặt prompt. Dù AI giúp tăng tốc đáng kể việc xác định *vì sao* một mutant sống sót, khi soi kỹ em nhận thấy output của nó có nhiều lỗi — từ sai sót kỹ thuật đến các giả định ngầm về môi trường.

2. (a) Lỗi sai về mặt dữ kiện trong output của AI

Lỗi rõ nhất xảy ra khi AI viết assertion cho mutant giỏ hàng. Đoạn code gốc `if (!userCarts[userId]) userCarts[userId] = [];` bị đột biến thành khởi tạo bằng một chuỗi giả. AI xác định đúng rằng cần khẳng định giỏ hàng rỗng, nhưng ban đầu lại gợi ý:

```
expect(res.body).toBe([]);
```

Đây là một lỗi dữ kiện căn bản trong kiểm thử JavaScript. Matcher `.toBe()` của Jest dùng `object.is` để so sánh bằng tuyệt đối. Vì mảng là kiểu tham chiếu (reference type), việc so mảng trong response với một mảng rỗng `[]` vừa khởi tạo sẽ **luôn luôn thất bại**, gây ra fail giả. AI đã không nhận ra đặc thù này của ngôn ngữ. Em phải tự sửa lại thành `expect(res.body).toEqual([])` (so sánh sâu) hoặc `expect(res.body.length).toBe(0)`.

3. (b) Thiếu các trường hợp biên

Khi kiểm tra lời giải của AI cho mutant biên của coupon (`usage_count >= max_uses` bị đột biến thành `>`), AI đưa được một test case chạm đúng giá trị biên: đặt số lần dùng bằng mức tối đa cho phép rồi thử áp coupon thêm một lần nữa.

Tuy nhiên, AI hoàn toàn bỏ sót các trường hợp biên xảy ra đồng thời (race condition). Trong môi trường EShop thực tế, hai request API có thể cùng áp một coupon tại cùng một mili-giây. Assertion do AI sinh ra ngầm giả định môi trường chạy tuần tự, đơn luồng và các test được cô lập tuyệt đối. Nó không hề gợi ý bọc test trong `Promise.all()` để mô phỏng các request đồng thời — vốn là một nguồn lỗi phổ biến ở điều kiện biên. Vì bỏ sót trường hợp này, test của AI tuy giết được mutant đơn giản nhưng vẫn để hệ thống hờ với race condition ngoài thực tế.

4. (c) Các giả định ngầm

AI đưa ra một giả định ngầm rất lớn về trạng thái môi trường test: nó cho rằng cơ sở dữ liệu trong bộ nhớ (`userCarts` và `orders`) hoàn toàn không lưu trạng thái và tự động reset giữa các lần chạy test.

Khi sinh assertion cho phần kiểm tra coupon, AI đưa các khối `it(...)` độc lập và mặc định rằng chúng sẽ pass hoàn hảo. Nó âm thầm bỏ qua các hook `beforeEach()` / `afterEach()` cần thiết để dọn sạch dữ liệu. Vì bộ test Jest của nhóm em chạy tuần tự, trạng thái còn sót lại từ các test coupon trước đã khiến những test do AI sinh ra fail bất ngờ. Ngoài ra, trong lúc xử lý các lần crash SQLite, AI còn ngầm giả định `process.env.JEST_WORKER_ID` luôn tồn tại, bỏ qua giai đoạn dry-run chạy in-band của Stryker.

5. (d) Những phát biểu quá tự tin

Xuyên suốt quá trình tương tác, AI thể hiện mức độ tự tin thái quá. Sau khi sinh test biên cho logic coupon, AI khẳng định:

"By adding this boundary assertion, your test suite will 100% kill the mutant and guarantee that your coupon validation logic is completely bulletproof."

Phát biểu này quá tự tin và dễ gây hiểu lầm nguy hiểm. Dù assertion đó có giết được đúng một mutant Stryker cụ thể, việc giết một mutant **không đồng nghĩa** với "logic bất khả xâm phạm". Mutation testing chỉ đo độ nhạy của bộ test đối với các thay đổi cú pháp định sẵn; nó không chứng minh được sự vắng mặt của lỗi kiến trúc, lỗ hổng bảo mật (ví dụ IDOR khi áp coupon) hay các thiếu sót về logic nghiệp vụ. AI đã đánh đồng "giết một mutant" với "đạt chất lượng phần mềm hoàn hảo".

10.3. [AI-03] Tuyên bố sử dụng AI

Nhóm 08 cam kết sử dụng công cụ AI (ChatGPT / Claude) một cách có trách nhiệm:

- AI chỉ được dùng để hỗ trợ giải thích mutant diff và gợi ý khung câu lệnh assertion.
- Không dùng AI để tự động tạo ra số liệu thí nghiệm giả.
- Mọi dòng code, cấu hình Jest/Stryker và chỉ số Mutation Score trong báo cáo này đều do nhóm em **tự thực thi và kiểm chứng 100% trên máy thật**.

10.4. [AI-04] Bài suy ngẫm (Reflective Statement)

Suy ngẫm về kiểm thử phần mềm có sự hỗ trợ của AI

Tác giả: Nhóm 08 (CS423 / CSC15003 — Software Testing)

Quá trình làm việc với các công cụ AI như ChatGPT và Claude trong Seminar T10 về Mutation Testing đã cho nhóm em nhiều góc nhìn giá trị về bức tranh kỹ thuật phần mềm hiện đại có sự tham gia của AI. Ban đầu, nhóm em xem các mô hình ngôn ngữ lớn như "cỗ máy sinh lời giải" có thể dễ dàng và các mutant còn sống và viết ra bộ test hoàn hảo. Nhưng sau khi kiểm chứng thực nghiệm nghiêm túc bằng Stryker Mutator và Jest, quan điểm của nhóm em đã chuyển từ *phụ thuộc thụ động* sang *đánh giá phân biện*.

Bài học quan trọng nhất là: AI có khả năng nhận diện mẫu cú pháp rất mạnh, nhưng lại thiếu nhận thức thật sự về ngữ cảnh lúc chạy (runtime context). AI rất giỏi giải thích *vì sao* một mutant sống sót khi đọc code diff, nhưng liên tục vấp ở những đặc thù thực thi của JavaScript — như gợi ý so sánh tham chiếu `.toBe([])` cho assertion mảng, hay ngầm giả định test chạy trong môi trường không lưu trạng thái. Nếu nhóm em cứ mù quáng copy-paste output

của AI mà không chạy Stryker tại chỗ, bộ test của nhóm em hẳn đã đầy lỗi fail giả và test chậm chạp (flaky). Hơn nữa, những khẳng định quá tự tin của AI rằng giết được mutant là "phần mềm bất khả xâm phạm" đã cho thấy rõ rủi ro của thiên kiến tự động hóa (automation bias) ở người kỹ sư.

Cuối cùng, seminar này chứng minh rằng AI là một trợ thủ mạnh mẽ cho việc tổng hợp test, nhưng sự giám sát của con người là không thể thay thế. Mutation testing đóng vai trò như một "trọng tài khách quan": một assertion do AI sinh ra chỉ có giá trị khi nó thực sự giết được mutant trong một sandbox chạy thật mà không gây tác dụng phụ. Là những kỹ sư phần mềm tương lai, nhóm em xác định phải theo tư duy **"AI hỗ trợ — con người kiểm chứng"**: tận dụng AI để soạn nháp nhanh, nhưng luôn giữ trách nhiệm kỹ thuật tuyệt đối thông qua kiểm thử thực nghiệm.

11. Chương 11: Bảng phân công & Đóng góp công việc (Project Contribution Statement)

Người phụ trách: 23127195 - Trần Mạnh Hùng

Bảng đóng góp công việc của 3 thành viên Nhóm 08 (được chuyển đổi từ Template chuẩn của môn học):

MSSV	Họ và tên	Vai trò chính	Hạng mục phụ trách	Tỷ lệ đóng góp	Căn cứ đánh giá
23127195	Trần Mạnh Hùng	Nhóm trưởng / DevOps	Setup môi trường Jest & Stryker, refactor <code>server.js</code> , thực nghiệm Auth API module, viết <code>stryker.auth.config.mjs</code> , soạn slide, quay video	33.3%	Hoàn thành đúng tiến độ, đóng góp mã nguồn cấu hình lõi.
23127060	Ninh Văn Khải	QA / AI Specialist	Phân tích surviving mutants Order/Cart module, thực nghiệm AI-Augmented workflow, thiết kế Activity "Kill the Mutant", biên soạn bộ AI Audit Pack (<code>AI-02</code> , <code>AI-03</code> , <code>AI-04</code>), fix SQLite <code>:memory:</code> , quay video	33.4%	Hoàn thành báo cáo AI Audit chất lượng cao, diệt mutant Order đạt yêu cầu
23127259	Nguyễn Tấn Thắng	Tech Writer / Tester	Thực nghiệm Product/Admin API module (đạt 100%), tổng hợp User Guide, phân tích Failure Modes của Stryker & AI, chuẩn bị báo cáo tuần và đóng gói, quay video	33.3%	Hoàn thành xuất sắc module Product 100%, biên soạn tài liệu chẵn chu.

Xác nhận của nhóm: Cả 3 thành viên đã làm việc bình đẳng, hợp tác chặt chẽ và đóng góp khối lượng công việc đồng đều nhau (100% tổng khối lượng).

12. Chương 12: Tài liệu tham khảo (References)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

1. **Stryker Mutator Official Documentation.** (2024). *StrykerJS: Mutation testing for JavaScript and TypeScript*. Available at: <https://stryker-mutator.io/docs/>
2. **Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Traon, Y. L., & Harman, M.** (2019). *Mutation Testing Advances: An Analysis and Survey*. *Advances in Computers*, Vol. 112, pp. 275-378.
3. **Offutt, A. J., & Untch, R. H.** (2001). *Mutation 2000: Uniting the Orthogonal*. In *Mutation Testing for the New Century*, Springer, pp. 34-44.
4. **Petrović, G., & Ivanković, M.** (2018). *State of Mutation Testing at Google*. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 163-171.
5. **Khoa Công nghệ Phần mềm - FIT@HCMUS.** (2026). *Seminar Track Briefing & Topic T10 Briefing: Mutation Testing & Test Effectiveness*. CS423 / CSC15003 Software Testing.

1. [Michael Feathers - Characterization Testing ↵](#)